QML

Quantum Machine Learning

A step-by-step chapter: from gradient descent to a QCNN in CUDA-Q

What you will be able to do after this chapter

- compute losses, gradients, metrics and their statistical uncertainty quickly and reliably;
- work with qubit states, Pauli operators, rotation and controlled gates, tensor products and entanglement;
- encode classical data (basis, angle, amplitude), evaluate quantum kernels and the outputs of small variational circuits;
- derive and apply the parameter-shift rule, and count the cost of exact gradients for shared and controlled parameters;
- read the QCNN of `QML.ipynb` line by line, count its layers, parameters and gates, and interpret its results honestly;
- reason about barren plateaus, noise, shots, simulators, GPUs, classical baselines and the limits of QML;
- solve every type of question in the accompanying 80-question QML assessment.

Companion material in the same repository: `presentations/QML_Lecture.pdf` (slides), `QML.ipynb` (CUDA-Q notebook) and `assessments/` (question paper and answer book).

Contents

QML — Quantum Machine Learning **3**

1 Learning from data: losses, gradients and neural networks . . . 3

1.1 The four ingredients . . . 3

1.2 Gradient descent and the chain rule . . . 4

1.3 Non-linearity . . . 4

2 Linear algebra and probability for QML . . . 4

3 Evaluating models honestly . . . 5

4 Qubits, measurement and the Bloch sphere . . . 5

5 Gates, tensor products and entanglement . . . 6

6 Encoding classical data . . . 6

7 Quantum kernels . . . 7

8 Variational models and hybrid training . . . 7

9 The parameter-shift rule . . . 7

10 The QCNN of `QML.ipynb` . . . 8

11 Trainability, shots and noise . . . 8

12 Other QML algorithms (theory only in this repository) . . . 9

13 CUDA-Q, simulation targets and performance . . . 9

14 Benchmarks and the question of advantage . . . 9

15 Problem-solving strategies and question map . . . 10

16 Practice problems . . . 10

QML — Quantum Machine Learning

How to use this chapter

The chapter is built as a staircase. Sections 1–3 are *classical machine learning and its mathematics*. Sections 4–5 are *quantum computing* (states, measurement, gates, entanglement). Sections 6–10 are *QML itself* (encoding, kernels, variational models, gradients, the QCNN of the notebook). Sections 11–14 are the *advanced* material and the *implementation* (trainability, noise, other algorithms, CUDA-Q, benchmarking). Section 15 collects strategies and a map from every assessment question to the section that teaches it; Section 16 has practice problems with answers.

Statements are marked as *implemented in the repository*, *theory only*, or *illustrative experiment* (run for the lecture with `presentations/qml_source/make_figures.py`). "Previously recorded" results are the outputs stored in `QML.ipynb`; "re-executed" results come from running the same code again (they agree exactly).

Definition: Conventions used everywhere in this chapter

- Qubit 0 is written first: $|q_0 q_1 \ldots\rangle$; two-qubit matrices use the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.
- $R_P(\theta) = e^{-i\theta P/2} = \cos\frac{\theta}{2} I - i \sin\frac{\theta}{2} P$ for $P \in \{X, Y, Z\}$; $R_Y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$.
- $\langle M\rangle = \langle\psi|M|\psi\rangle$; $\langle Z\rangle = P(0) - P(1)$; Bloch vector $(\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$.
- Labels $y = \pm 1$ with prediction $\operatorname{sign} f$ (or $y \in \{0, 1\}$ with probabilities); MSE $\mathcal{L} = \frac{1}{M}\sum (f_m - y_m)^2$.
- `QML.ipynb`: 2×4 images, 8 qubits, $R_Y(\pi x_i)$ encoding, QCNN with 3 layers $\times$ 4 shared parameters, readout $\langle Z_7\rangle$, COBYLA with 100 loss evaluations, 20 training and 20 test images, seed 42.

1 Learning from data: losses, gradients and neural networks

1.1 The four ingredients

A supervised learning problem has data $(\mathbf{x}_m, y_m)_{m=1}^M$, a model $f(\mathbf{x}; \boldsymbol{\theta})$, an optimizer, and a loss

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{M} \sum_{m=1}^{M} \ell(f(\mathbf{x}_m; \boldsymbol{\theta}), y_m).$$

Classification predicts a class, *regression* a number; *unsupervised* learning has no labels (clustering, PCA); *reinforcement* learning learns from rewards.

Definition: common losses

MSE $(f-y)^2$; binary cross-entropy $-[y\ln p+(1-y)\ln(1-p)]$; cross-entropy $-\ln p_{\text{true}}$ with $p_k=e^{z_k}/\sum_j e^{z_j}$; hinge $\max(0,1-yf)$.

1.2 Gradient descent and the chain rule

$\boldsymbol{\theta}\leftarrow\boldsymbol{\theta}-\eta\nabla\mathcal{L}$. For $\mathcal{L}=\frac{1}{2}\boldsymbol{\theta}^{\mathsf{T}}H\boldsymbol{\theta}$ the iteration is $\boldsymbol{\theta}\leftarrow(I-\eta H)\boldsymbol{\theta}$ and converges iff $0<\eta<2/\lambda_{\max}$. Momentum: $v\leftarrow\beta v+g$, $\boldsymbol{\theta}\leftarrow\boldsymbol{\theta}-\eta v$. For linear regression $\nabla\frac{1}{M}\|X\mathbf{w}-\mathbf{y}\|^2=\frac{2}{M}X^{\mathsf{T}}(X\mathbf{w}-\mathbf{y})$.

Worked example 1.1: one step and one neuron

Step 1. $\mathcal{L}=(w_1-3)^2+2(w_2+1)^2$, $\mathbf{w}=(0,0)$, $\eta=0.1$: $\nabla\mathcal{L}=(-6,4)$, new $\mathbf{w}=(0.6,-0.4)$, $\mathcal{L}$: $11\to 6.48$.

Step 2. $f=\sigma(wx+b)$, $L=(f-y)^2$, $x=2$, $w=0.5$, $b=-1$, $y=1$: $z=0$, $f=\frac{1}{2}$, $\partial_w L=2(f-y)\sigma'(z)x=2(-\frac{1}{2})(\frac{1}{4})(2)=-\frac{1}{2}$.

Step 3. Sigmoid + cross-entropy: $\partial L/\partial z=p-y$ (the σ' cancels). A ReLU with negative input passes no gradient.

Key result: activation derivatives

$\sigma'=\sigma(1-\sigma)$, $\tanh'=1-\tanh^2$, $\text{ReLU}'=[z>0]$, $\frac{d}{dz}\ln(1+e^z)=\sigma(z)$. A dense layer $n_{\text{in}}\to n_{\text{out}}$ has $n_{\text{in}}n_{\text{out}}+n_{\text{out}}$ parameters ($8\to 4\to 1$: 41).

Common trap

Moving *along* the gradient; forgetting the factor 2 of a square or the inner derivative x; dividing by $\log_{10}$ instead of ln; summing instead of averaging; using the wrong class's probability in a cross-entropy.

1.3 Non-linearity

XOR is not linearly separable: from $g(0,1),g(1,0)>0$ and $g(0,0),g(1,1)<0$ one gets both $w_1+w_2+2b>0$ and <0. The same argument shows that the clean line images of `QML.ipynb` are not linearly separable: the horizontal pairs in columns 1–2 of both rows light exactly the same four pixels as the vertical lines in columns 1 and 2.

2 Linear algebra and probability for QML

Key result: toolkit

- Eigen-decomposition: PCA directions are eigenvectors of the covariance; explained fraction $\lambda_1/\operatorname{tr}C$.
- Kronecker product $A\otimes B=(a_{ij}B)$; $(A\otimes B)(C\otimes D)=AC\otimes BD$; $\operatorname{tr}(A\otimes B)=\operatorname{tr}A\operatorname{tr}B$; not commutative.
- Kernels: $k(\mathbf{u},\mathbf{v})=e^{-\|\mathbf{u}-\mathbf{v}\|^2/(2\sigma^2)}$; a valid kernel has PSD Gram matrices.
- Bayes: $P(D\mid+)=P(+\mid D)P(D)/P(+)$. Binomial standard error $\sqrt{a(1-a)/M}$.

Worked example 2.2: PCA and Bayes

$C = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$: $\lambda = 3, 1$; first component $(1,1)/\sqrt{2}$ explains 75 %. Recall 0.9, false-positive rate 0.05, prevalence 0.01: $P(D \mid +) = 0.009/0.0585 = 15\,\%$.

3 Evaluating models honestly

Definition: metrics

Accuracy $\frac{TP+TN}{\text{all}}$; precision $\frac{TP}{TP+FP}$; recall $\frac{TP}{TP+FN}$; $F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$.

Method: a trustworthy evaluation

1. split first; fit preprocessing on the training split only; tune on a validation split; touch the test set once;
2. report the test-set size and SE $= \sqrt{a(1-a)/M}$ (20 images, $a = 0.85$: ± 8 points);
3. compare with strong classical baselines on the same data; diagnose gap (overfitting) vs. both-high (underfitting).

Worked example 3.3: the re-executed QCNN on 20 test images

TP 9, FN 1, FP 2, TN 8: accuracy 0.85, precision $9/11 = 0.818$, recall 0.900, $F_1 = 18/21 = 0.857$ (the arithmetic mean 0.859 is *not* F1).

Common trap

Reporting accuracy on imbalanced data; choosing hyper-parameters on the test set; confusing training MSE with accuracy (the recorded run has MSE 0.67 at 90 % accuracy because its outputs are small, $|\langle Z_7 \rangle| \approx 0.2$).

4 Qubits, measurement and the Bloch sphere

$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, $|\alpha|^2 + |\beta|^2 = 1$. Measurement gives 0 with probability $|\alpha|^2$.

Key result: expectation values of one qubit

$\langle Z \rangle = |\alpha|^2 - |\beta|^2$, $\langle X \rangle = 2\,\mathrm{Re}(\alpha^*\beta)$, $\langle Y \rangle = 2\,\mathrm{Im}(\alpha^*\beta)$. For $\cos\frac{\vartheta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\vartheta}{2}\,|1\rangle$: $\vec{r} = (\sin\vartheta\cos\varphi, \sin\vartheta\sin\varphi, \cos\vartheta)$.

Worked example 4.4: a phase changes $\langle X \rangle$, not $\langle Z \rangle$

$\frac{1}{2}\,|0\rangle + \frac{\sqrt{3}}{2}e^{i\pi/4}\,|1\rangle$: $\langle Z \rangle = -\frac{1}{2}$, $\langle X \rangle = \frac{\sqrt{3}}{2}\cos\frac{\pi}{4} = \frac{\sqrt{6}}{4}$.

Key result: shots

S shots estimate $\langle Z \rangle$ with standard error $\sqrt{(1-\langle Z \rangle^2)/S}$. 620 zeros in 1000 shots: 0.24 ± 0.031.

Common trap

Forgetting the half angle ($\vartheta = \frac{\pi}{4}$ means amplitudes $\cos\frac{\pi}{8}$, $\sin\frac{\pi}{8}$); treating a global phase as observable; concluding that $R_X(\theta)\,|0\rangle$ and $R_X(-\theta)\,|0\rangle$ are equal because their Z-statistics agree.

5 Gates, tensor products and entanglement

Key result: gate facts

$X^2 = Y^2 = Z^2 = I$; $XY = iZ = -YX$ (cyclic); $(aX+bY+cZ)^2 = (a^2+b^2+c^2)I$. $HZH = X$, $SXS^\dagger = Y$, $R_P(\pi) = -iP$. $R_Y(\theta)^\dagger Z R_Y(\theta) = \cos\theta Z - \sin\theta X$. CNOT: $Z_t \to Z_c Z_t$, $X_t \to X_t$, $X_c \to X_c X_t$. Controlled-$R_Z(\theta) = \mathrm{diag}(1, 1, e^{-i\theta/2}, e^{i\theta/2})$ (control qubit 0).

Definition: product state and entanglement

$\sum c_{ab}\,|ab\rangle$ is a product state iff $c_{00}c_{11} - c_{01}c_{10} = 0$. Purity of the reduced state $\mathrm{tr}\,\rho_1^2 = 1$ (product) down to $\frac{1}{2}$ (maximally entangled).

Worked example 5.5: $R_Y(\theta)$ then CNOT

$\cos\frac{\theta}{2}\,|00\rangle + \sin\frac{\theta}{2}\,|11\rangle$: $\langle Z_1\rangle = \cos\theta$, $\langle Z_0 Z_1\rangle = 1$, $\mathrm{tr}\,\rho_1^2 = \frac{1+\cos^2\theta}{2}$, maximal entanglement at $\theta = \frac{\pi}{2}$. A local $R_Y(\theta)$ on half of $|\Phi^+\rangle$ leaves $\langle Z_1\rangle = 0$ for every θ.

Worked example 5.6: a two-parameter circuit

$R_X(\theta_2)R_Y(\theta_1)\,|0\rangle$: $(0,0,1) \to (\sin\theta_1, 0, \cos\theta_1) \to z = \cos\theta_1\cos\theta_2$. A parameter in a single rotation gate gives $f = a + b\cos\theta + c\sin\theta$ (period 2π).

Key result: memory

n qubits: 2^n amplitudes, $16 \cdot 2^n$ bytes (fp64). 30 qubits: 16 GiB; 80 GB GPU: 32 qubits; four of them: 34.

6 Encoding classical data

encoding	state	qubits	gates	remark
basis	$\lvert b_1 \dots b_n\rangle$	1 per bit	$\le n$	binary only
angle	$\bigotimes R_Y(x_i)\,\lvert 0\rangle$	d	d, depth 1	`QML.ipynb`: $R_Y(\pi x_i)$
amplitude	$\sum_j x_j\,\lvert j\rangle\,/\,\lVert\mathbf{x}\rVert$	$\lceil\log_2 d\rceil$	$O(d)$ generic	norm lost
ZZ map	$U_\Phi(\mathbf{x})H^{\otimes n}\cdots$	d	entangling	theory only

Worked example 6.7: the three encodings

Pixel $x = \frac{1}{3}$: $P(1) = \sin^2\frac{\pi}{6} = \frac{1}{4}$, $\langle Z\rangle = \cos\frac{\pi}{3} = \frac{1}{2}$. $\mathbf{x} = (1,2,4,2)$: norm 5, amplitudes $(0.2, 0.4, 0.8, 0.4)$, $P(q_0 = 1) = 0.80$, $P(q_1 = 1) = 0.32$. 256 features of 8 bits: basis 2048, angle 256, amplitude 8 qubits.

Key result: data re-uploading

With L encoding gates $e^{-ixP/2}$, $f(x)$ is a Fourier series with frequencies $0, \dots, L$.

7 Quantum kernels

$k(\mathbf{x}, \mathbf{x}') = |\langle\psi(\mathbf{x})|\psi(\mathbf{x}')\rangle|^2 = \mathrm{tr}[\rho(\mathbf{x})\rho(\mathbf{x}')]$ is symmetric and PSD; estimates from shots may not be. Estimate it by compute–uncompute ($P(0\dots0)$) or by the swap test, $P(0) = \frac{1}{2}(1+k)$.

Key result: the notebook's encoding gives a classical kernel

$k(\mathbf{x}, \mathbf{x}') = \prod_i \cos^2 \frac{\pi(x_i - x_i')}{2}$. Example $\mathbf{x} = (1, 0.5)$, $\mathbf{x}' = (0.5, 0)$: $k = \frac{1}{4}$. For independent uniform features $\mathbb{E}[k] = (\frac{1}{2} + \frac{2}{\pi^2})^n \approx 0.70^n$: kernel values concentrate exponentially.

Worked example 7.8: kernel SVM on the notebook's data (illustrative experiment)

A soft-margin SVM with this kernel, trained on the same 20 images, classifies all 20 test images correctly and 95.2 % of 2000 fresh ones (QCNN: 80.2 %). The kernel was computed from CUDA-Q states and agrees with the formula to 10^{-15} — a classical computation, so this is no quantum advantage. Training needs $\binom{M}{2}$ kernel circuits (190 for $M = 20$).

8 Variational models and hybrid training

$f(\mathbf{x}; \boldsymbol{\theta}) = \langle\psi(\mathbf{x})|U(\boldsymbol{\theta})^\dagger M U(\boldsymbol{\theta})|\psi(\mathbf{x})\rangle$, prediction sign f, loss computed classically, parameters updated by a classical optimizer: *classical input* → *encoding* → *PQC* → *measurement* → *classical loss* → *optimizer* → *update*.

Worked example 8.9: a two-qubit VQC (Heisenberg picture)

$R_Y(x_0) \otimes R_Y(x_1)$, $\mathrm{CNOT}_{0\to1}$, $R_Y(\theta)$ on qubit 1, $f = \langle Z_1\rangle$: $Z_1 \to \cos\theta Z_1 - \sin\theta X_1 \to \cos\theta Z_0 Z_1 - \sin\theta X_1$, so $f = \cos\theta\cos x_0\cos x_1 - \sin\theta\sin x_1$. At $\mathbf{x} = (0, \frac{\pi}{3})$, $\theta = \frac{\pi}{6}$: $f = 0$, $f(\theta \pm \frac{\pi}{2}) = \mp 1$, $f' = -1$, and with $y = 1$: $\partial_\theta (f - y)^2 = 2$.

Key result: cost of training in the notebook

One loss evaluation runs one circuit per training image: $100 \times 20 + 20 = 2020$ circuits, ≈ 1.7 ms each on `qpp-cpu`: ≈ 3.4 s (recorded). MSE 0.505 and accuracy 75 % for outputs $(0.6, -0.2, 0.9, 0.1)$, labels $(+, -, +, -)$.

9 The parameter-shift rule

Key result: two-term rule

For a parameter in exactly one gate $e^{-i\theta P/2}$ ($P^2 = I$): $f = a + b\cos\theta + c\sin\theta$ and $f'(\theta) = \frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$ exactly.

Common trap

Shared parameters: $\theta_{1,1}$ of the QCNN appears in 7 gates; shifting it once is wrong — shift every occurrence and add. *Controlled rotations*: generator eigenvalues $0, \pm\frac{1}{2}$; use $f' = d_+[f(\theta+\frac{\pi}{2}) - f(\theta-\frac{\pi}{2})] - d_-[f(\theta+\frac{3\pi}{2}) - f(\theta-\frac{3\pi}{2})]$, $d_\pm = \frac{\sqrt{2}\pm 1}{4\sqrt{2}}$. For $f = \cos\frac{\theta}{2}$ at $\theta = \pi$ the two-term rule gives -0.707, the truth is -0.5.

Key result: cost of one exact QCNN gradient

Per image $4n_{\text{conv}} + 8n_{\text{pool}} = 44 + 56 = 100$ circuits; for 20 images 2000 — the size of the whole COBYLA run.

10 The QCNN of `QML.ipynb`

Definition: layout and blocks

`qcnn_layout(n)`: in each layer, convolution blocks on pairs $(0,1),(2,3),\ldots$ then $(1,2),(3,4),\ldots$ of the active qubits; pooling blocks (source a, sink b) on $(0,1),(2,3),\ldots$; sinks survive; an odd last qubit moves on. Convolution: $\text{CNOT}_{a\to b}(R_Y(\theta_{\ell,1}) \otimes R_Y(\theta_{\ell,2}))$. Pooling: $|1\rangle\langle 1|_a \otimes R_Z(\theta_{\ell,3}) + |0\rangle\langle 0|_a \otimes R_X(\theta_{\ell,4})$ (code: `rz.ctrl; x; rx.ctrl; x`), then a is discarded (deferred measurement).

Key result: counts

Layers $\lceil \log_2 N \rceil$, parameters $4\lceil \log_2 N \rceil$, gates $N + 3n_{\text{conv}} + 4n_{\text{pool}}$. $N = 8$: 11 conv, 7 pool, 69 gates, depth 21, output qubit 7. $N = 16$: 26, 15, 154 gates. $N = 20$: 5 layers, 20 parameters.

Worked example 10.10: the recorded run

CUDA-Q 0.16.0, `qpp-cpu`, no GPU: loss 1.1214 → 0.6704, training accuracy 90 %, test accuracy 85 %, 3.4 s. Re-executed: identical; independent NumPy simulation agrees to 7×10^{-16}.

11 Trainability, shots and noise

Key result: barren plateaus

Deep random circuits: $\text{Var}[\partial_k C] \sim 2^{-n}$, so $\sim 2^n$ shots are needed per gradient component (20 qubits: 10^6). Shallow circuits with local costs: polynomial decay; global costs: exponential. Gradient-free optimizers do not escape. The QCNN is provably free of barren plateaus (Pesah et al. 2021).

Key result: noise

Depolarising noise shrinks $\langle Z \rangle$ roughly by $(1-p)^G$ ($0.99^{69} \approx 0.50$); signs survive longer than margins. Measured (illustrative): the trained QCNN keeps 85 % accuracy up to $p = 0.1$ per gate, but its mean $|\langle Z \rangle|$ falls from 0.171 to 0.003, which would need $\sim 10^5$ shots per image to read.

12 Other QML algorithms (theory only in this repository)

- **QSVM**: classical convex dual with a quantum kernel; $O(M^2)$ kernel circuits; no barren plateaus; concentration risk.
- **QGAN**: generator PQC vs. discriminator; equilibrium value $-\ln 4$.
- **qPCA**: density-matrix exponentiation + phase estimation; needs many copies, low rank, efficient state access; dequantized classically.
- **Clustering**: distances from overlaps, $\|\mathbf{a}-\mathbf{b}\|^2 = 2 - 2\mathbf{a}\cdot\mathbf{b}$; swap test $P(0) = 0.9 \Rightarrow 0.46$.
- **Quantum RL**: PQC policies or Q-functions; Q-learning update $Q \leftarrow Q + \alpha(r + \gamma \max Q' - Q)$ (e.g. $2 \to 3.3$).
- **Quantum data**: the strongest known advantages concern learning from quantum states, not small classical datasets.

13 CUDA-Q, simulation targets and performance

Key result: targets

`qpp-cpu` (recorded run), `nvidia` (one GPU, chosen by `"auto"` if visible), `nvidia+mgpu` (one state vector over several GPUs via MPI), `nvidia+mqpu` (many circuits in parallel — the natural fit for QML), `density-matrix-cpu` (noise, $16 \cdot 4^N$ bytes). `QML.ipynb` uses neither `mgpu` nor `mqpu`; no GPU path was executed.

Worked example 13.11: reading counts

`{00:400, 01:100, 10:300, 11:200}` (qubit 0 first): $\langle Z_0 \rangle = 0$, $\langle Z_1 \rangle = 0.4$, $\langle Z_0 Z_1 \rangle = 0.2$.

Worked example 13.12: time

15 qubits, 0.35 s per circuit (README), 16 images $\times$ 60 evaluations: 960×0.35 s ≈ 5.6 min.

14 Benchmarks and the question of advantage

On the same 20 training images (illustrative experiments): logistic regression on pixels 45 % test (46.5 % on 2000 fresh images — not linearly separable), hand-made rule 100 %, angle-kernel SVM 100 % (95.2 %), QCNN 85 % (80.2 %).

Common trap

Claiming quantum advantage from an accuracy above chance, from beating a weak baseline, or from a kernel that is classically computable. Advantage needs strong baselines, equal budgets, error bars and all costs counted (encoding, shots, optimizer steps).

15 Problem-solving strategies and question map

Method: five habits

1. **Convention first**: which qubit is written first? Which half-angle? Which positive class?
2. **Bloch or Heisenberg**: follow the Bloch vector for one qubit; push observables back through gates for small circuits.
3. **Products factorise**: product states give product kernels and product expectation values.
4. **Count occurrences**, not parameters, for gradients; count blocks from the layout for gates.
5. **Separate** recorded from re-executed from illustrative results; attach an error bar to every accuracy.

Section	Assessment questions
1 Losses, gradients, networks	1, 2, 6, 7, 8, 13, 14, 15, 17, 18, 19, 20
2 Linear algebra, probability	4, 5, 9, 10, 11, 22, 29
3 Evaluation	3, 11, 12, 16, 68, 79, 80
4 Qubits, measurement, Bloch	21, 23, 27, 30, 33, 36, 37
5 Gates, entanglement	24, 25, 26, 28, 31, 32, 34, 35, 38, 39, 40
6 Encoding	41, 42, 43, 54, 55
7 Quantum kernels	9, 44, 45, 46, 58, 70, 74
8 Variational models, hybrid training	34, 52, 53, 56, 59, 60
9 Parameter shift	47, 48, 57, 60, 63, 66
10 QCNN of `QML.ipynb`	48, 49, 50, 51, 53, 66, 68
11 Trainability, shots, noise	26, 27, 61, 62, 64
12 Other algorithms	56, 70, 71, 72, 73, 74
13 CUDA-Q, performance	33, 65, 67, 69, 75, 78
14 Benchmarks, advantage	44, 76, 77, 79, 80

16 Practice problems

Try each before looking at the answers on the next page. They use the same ideas as the assessment but different numbers.

P1. One gradient step on $\mathcal{L} = 2(w-1)^2$ from $w = 3$ with $\eta = 0.2$: new w and $\mathcal{L}$.

P2. Mean binary cross-entropy for $(p, y) = (0.7, 1)$ and $(0.4, 0)$.

P3. $TP = 30$, $FP = 20$, $FN = 10$: precision, recall, F1.

P4. $\langle Z \rangle$ and $\langle X \rangle$ of $\frac{\sqrt{3}}{2}|0\rangle + \frac{1}{2}|1\rangle$.

P5. Angle-encode the pixel $x = 0.5$ with $R_Y(\pi x)$: $P(1)$ and $\langle Z \rangle$.

P6. Amplitude-encode $(2, 1, 2, 0, 0, 0, 0, 4)$: number of qubits and $P(q_0 = 1)$ (qubit 0 first).

P7. The notebook's kernel for $\mathbf{x} = (0.2, 0.6)$, $\mathbf{x}' = (0.2, 0.1)$.

P8. $f(\theta + \frac{\pi}{2}) = -0.2$, $f(\theta - \frac{\pi}{2}) = 0.6$ for a single $R_Y(\theta)$: $f'(\theta)$.

P9. The notebook's QCNN for $N = 6$: layers, parameters, conv and pool blocks, gates.

P10. Circuits per image for one exact gradient of the $N = 6$ QCNN (two-term per R_Y, four-term per controlled rotation).

P11. Standard error of an accuracy of 0.75 on 40 test images.

P12. Largest fp64 state vector on a 40 GB GPU.

P13. Swap test with $P(0) = 0.6$: the fidelity kernel.

P14. Depolarising factor $(1 - p)^G$ for $p = 0.005$ and the 15-qubit QCNN ($G = 146$).

Answers to the practice problems

P1. $\mathcal{L}' = 4(w-1) = 8$; $w = 3 - 1.6 = 1.4$; $\mathcal{L} = 2(0.4)^2 = 0.32$.

P2. $\frac{1}{2}(-\ln 0.7 - \ln 0.6) = \frac{1}{2}(0.357 + 0.511) = 0.434$.

P3. $P = 0.6$, $R = 0.75$, $F_1 = 60/90 = 0.667$.

P4. $\langle Z \rangle = \frac{3}{4} - \frac{1}{4} = \frac{1}{2}$; $\langle X \rangle = 2 \cdot \frac{\sqrt{3}}{2} \cdot \frac{1}{2} = \frac{\sqrt{3}}{2}$.

P5. $P(1) = \sin^2 \frac{\pi}{4} = \frac{1}{2}$; $\langle Z \rangle = \cos \frac{\pi}{2} = 0$.

P6. Norm 5, 3 qubits; $q_0 = 1$ for indices 4–7: $P = 16/25 = 0.64$.

P7. $\cos^2 0 \cdot \cos^2 \frac{\pi}{4} = \frac{1}{2}$.

P8. $\frac{1}{2}(-0.2 - 0.6) = -0.4$.

P9. $6 \to 3 \to 2 \to 1$: 3 layers, 12 parameters; conv $5 + 2 + 1 = 8$, pool $3 + 1 + 1 = 5$; gates $6 + 24 + 20 = 50$.

P10. $4 \cdot 8 + 8 \cdot 5 = 72$.

P11. $\sqrt{0.75 \cdot 0.25/40} = 0.068$.

P12. $2^{31} \cdot 16 = 34.4$ GB fits, $2^{32} \cdot 16 = 68.7$ GB does not: 31 qubits.

P13. $2(0.6) - 1 = 0.2$.

P14. $0.995^{146} = e^{146 \ln 0.995} \approx 0.48$.

Summary of the chapter in ten lines

1. Learning = data + model + loss + optimizer; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}$; chain rule for gradients.
2. Evaluate on untouched test data, with error bars ($\sqrt{a(1-a)/M}$) and strong baselines; use P/R/F1 for imbalance.
3. Qubits: $\langle Z \rangle = |\alpha|^2 - |\beta|^2$; half angles; global phases invisible; shots give $\pm 1/\sqrt{S}$.
4. $R_Y^\dagger Z R_Y = \cos\theta Z - \sin\theta X$; CNOT moves $Z_t \to Z_c Z_t$; $\det C = 0 \iff$ product state.
5. Encodings: basis (bits), angle (d qubits, depth 1), amplitude ($\log_2 d$ qubits, deep).
6. Fidelity kernels are PSD; product encodings give classical product kernels that concentrate like 0.70^n.
7. Hybrid loop: encode → PQC → measure → classical loss → optimizer → update.
8. Parameter shift is exact for single Pauli rotations; shared parameters per occurrence; controlled rotations four terms.
9. QCNN: $\lceil \log_2 N \rceil$ layers, 4 shared parameters each; $N = 8$: 12 parameters, 69 gates, 90/85 % recorded.
10. Barren plateaus, noise, shots and data loading limit QML; no quantum advantage is claimed or shown here.